

LAPORAN JOBSHEET 10: STATICSIDEGENERATION MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

C. LANGKAH PRAKTIKUM

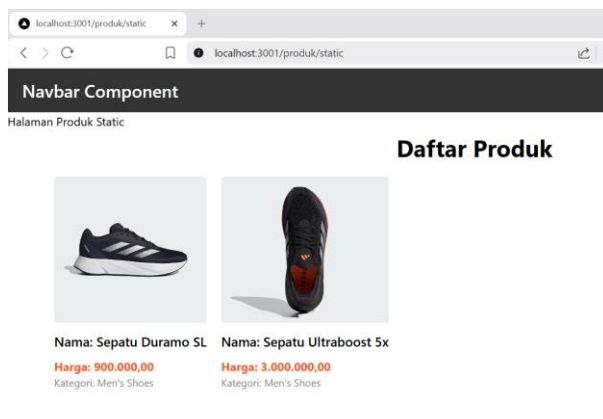
Bagian 1 – Setup Halaman Static

Buat file baru pada pages/products/static.tsx

Modifikasi file static.tsx

```
pemograman_framework - static.tsx
1  import TampilanProduk from "../views/product";
2  import { ProductType } from "../types/Product.type";
3
4  const halamanProdukStatic = (props:{products:ProductType[]}) => {
5    const {products} = props;
6    return (
7      <div>
8        <h1>Halaman Produk Static</h1>
9        <TampilanProduk products={products}/>
10      </div>
11    )
12  }
13
14  export default halamanProdukStatic;
15
16  export async function getStaticProps(){
17    const res = await fetch('http://127.0.0.1:3001/api/produk');
18    // const response: ProductType[] = await res.json();
19    const response: {data: ProductType[]} = await res.json();
20
21    console.log("Data produk yang diambil dari API:", response.data);
22    return {
23      props: {
24        products: response.data
25      }
26    }
27  }
28
```

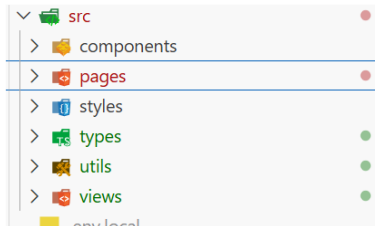
Output



Bagian 3 – Build Production Mode

Pindah beberapa folder diluar pages antara lain, folder Views, utils, styles dipindah di luar

folder src



alankan: npm run build

- Jalankan npm run dev dan pastikan ini jalan (jangan distop saat ngebuild), jadi

buka dua terminal

o Terminal 1 : jalankan aplikasi npm run dev

o Terminal 2 : build aplikasi

Hal ini dikarenakan aplikasi berjalan di local http://localhost:3000/api/produk (

jika berhasil maka akan muncul pesan route(pages))

```
Route (pages)
├─ /
├─ /_app
├─ /404
├─ /about
├─ f /api/hello
├─ f /api/produk
├─ o /auth/login
├─ o /auth/register
├─ o /blog
├─ o /blog/[slug]
├─ o /category/[...slug]
├─ o /produk
├─ o /produk/[id]
├─ f /produk/server
├─ ● /produk/static
├─ o /profil
├─ o /profil/edit
├─ o /setting/app
├─ o /shop/[...slug]
├─ o /user
├─ o /user/password
├─ o (Static) prerendered as static content
├─ ● (SSG) prerendered as static HTML (uses getStaticProps)
├─ f (Dynamic) server-rendered on demand
PS D:\pemogramaran_framework\Code\praktikum\my-app>
```

Jika berhasil: npm run start

Jika saat menjalankan npm run start muncul error :

```
PS D:\pemograman_framework\Code\praktikum\my-app> npm run start

> my-app@0.1.0 start
> next start

x Failed to start server
Error: listen EADDRINUSE: address already in use :::3000
    at <unknown> (Error: listen EADDRINUSE: address already in use :::3000)
    at new Promise (<anonymous>) {
  code: 'EADDRINUSE',
  errno: -4091,
  syscall: 'listen',
  address: ':::',
  port: 3000
}
PS D:\pemograman_framework\Code\praktikum\my-app>
```

Maka ada perintah npm run dev yang sedang jalan , matikan terlebih dahulu setelah lakukan perintah npm run start

```
PS D:\pemograman_framework\Code\praktikum\my-app> npm run start -- -p 3001

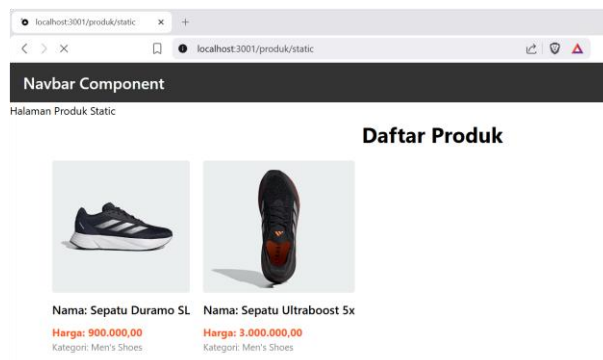
> my-app@0.1.0 start
> next start -p 3001

▲ Next.js 16.1.6
- Local:      http://localhost:3001
- Network:    http://172.16.0.2:3001

✓ Starting...
✓ Ready in 434ms
```

Untuk di kasus saya saya harus set port nya ke 3001 karena port 3000 sudah digunakan oleh java.exe seperti diatas.

Akses: <http://localhost:3000/products/static>



Bagian 4 – Pengujian Perubahan Data

Uji 1 – Tambah Data di Database

1. Buka database firebasenya

- Tambahkan produk baru di database.

Add a document

Parent path
/products

Document ID [?](#)

nuggh8wLOzGCjAmcRvqP

Field	Type
name	string
String	
Sepatu SL 72 OG	
image	string
String	
http	
price	number
Value	
1600000	
category	string
String	
sepatu lain	

... Add field

Cancel Save

products	nuggh8wLOzGCjAmcRvqP
products	nuggh8wLOzGCjAmcRvqP
products	nuggh8wLOzGCjAmcRvqP

Buka halaman:

- /products (CSR) → Data bertambah

localhost:3001/produk

Navbar Component

Daftar Produk



Nama: Sepatu Duramo SL
Harga: 900.000,00
Kategori: Men's Shoes

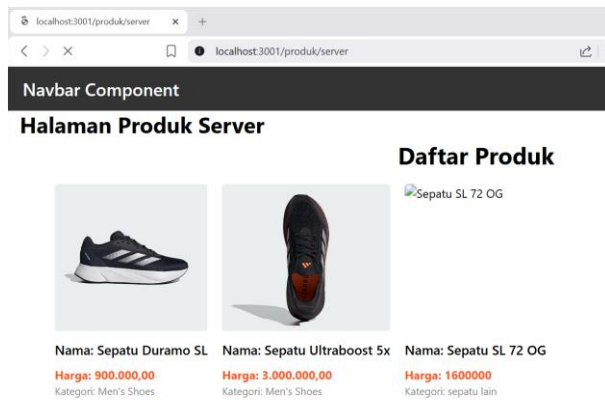


Nama: Sepatu Ultraboost 5x
Harga: 3.000.000,00
Kategori: Men's Shoes

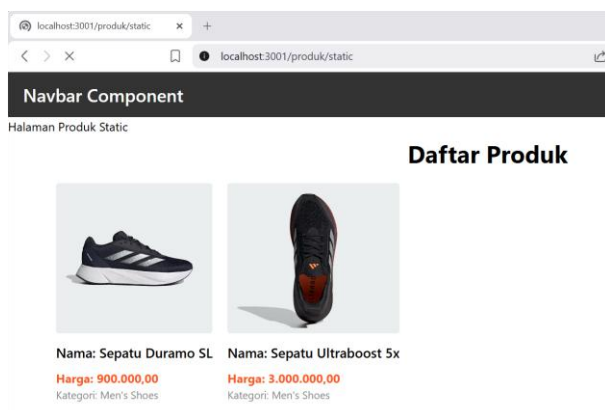


Nama: Sepatu SL 72 OG
Harga: 1600000
Kategori: sepatu lain

/products/server (SSR) → Data bertambah



/products/static (SSG) → Data tidak berubah



Uji 2 – Build Ulang

1. Jalankan kembali:

- npm run build

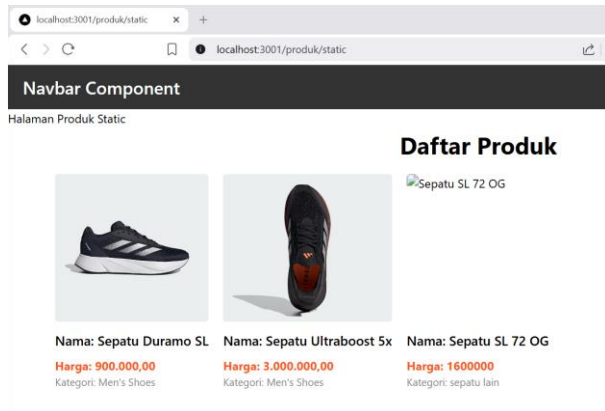
o lakukan secara bersamaan dengan npm run dev saat melakukan npm run build

- npm run start

o npm run dev stop terlebih dahulu setelah itu npm run start

2 Refresh halaman static → Data

- baru muncul



D. TUGAS PRAKTIKUM

Tugas Individu

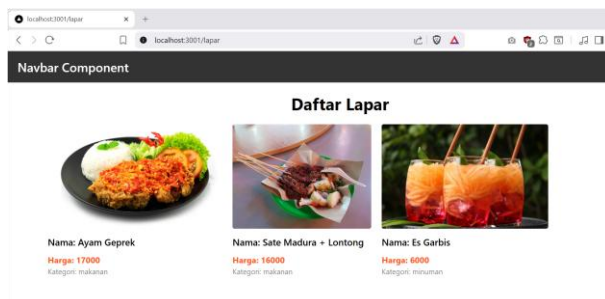
1. Buat 3 halaman:

o CSR /lapar

```

1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 import TampilanLapar from "../../views/lapar";
4 import useSWR from "swr";
5 import fetcher from "../../utils/swr/fetcher";
6 // const fetcher = (url: string) => fetch(url).then((res) => res.json());
7 const kategori = () => {
8   // const [isLogin, setIsLogin] = useState(false);
9   // const [push] = useRouter();
10  const [lapars, setLapars] = useState([]);
11  // Menggunakan SWR
12  const {data, error, isLoading} = useSWR("http://localhost:3001/api/lapar", fetcher);
13
14  // cek apakah data, error, atau isLoading sudah benar
15  console.log("Data:", data);
16  // console.log("Error:", error);
17  // console.log("Is Loading:", isLoading);
18
19  return (
20    <div className="container mx-auto p-4">
21      <TampilanLapar lapars={data?.data || []} isLoading={isLoading} />
22    </div>
23  );
24 };
25 export default kategori;

```

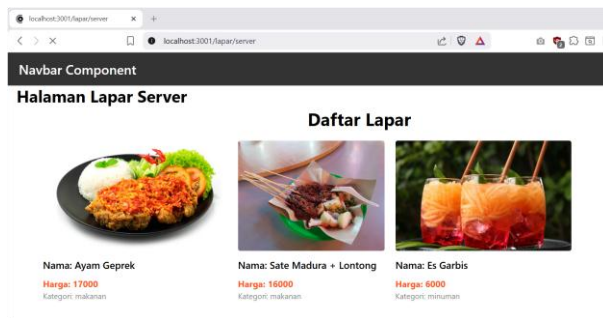


o SSR /lapar/server

```

1 import TampilanLapar from "../../views/lapar";
2 import { LaparType } from "../../types/Lapar.type";
3
4 const halamanLaparServer = (props: { lapars: LaparType[] }) => {
5   const { lapars } = props;
6   return (
7     <div>
8       <h1 className="font-bold text-3xl pl-4">Halaman Lapar Server</h1>
9       <TampilanLapar lapars={lapars}/>
10     </div>
11   )
12 }
13 export default halamanLaparServer;
14
15 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data lapar dari API sebelum merender halaman.
16 export async function getServerSideProps() {
17   const res = await fetch("http://localhost:3001/api/lapar");
18   const response = await res.json();
19   // console.log("Data lapar yang diambil dari API", response);
20   return {
21     props: {
22       lapars: response.data,
23     },
24   };
25 }
26

```

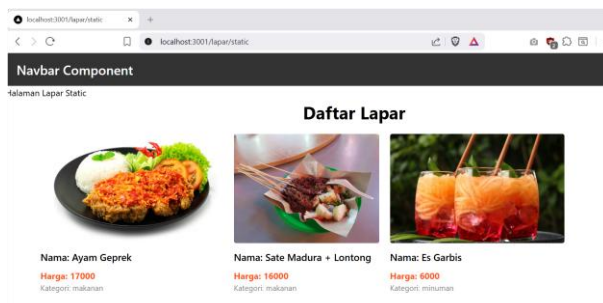


o SSG lapar/static

```

1 import TampilanLapar from "../../views/lapar";
2 import { LaparType } from "../../types/Lapar.type";
3
4 const halamanLaparStatic = (props: { lapars: LaparType[] }) => {
5   const { lapars } = props;
6   return (
7     <div>
8       <h1>Halaman Lapar Static</h1>
9       <TampilanLapar lapars={lapars}/>
10     </div>
11   )
12 }
13
14 export default halamanLaparStatic;
15
16 export async function getStaticProps(){
17   const res = await fetch('http://127.0.0.1:3001/api/lapar');
18   // const response: LaparType[] = await res.json();
19   const response: { data: LaparType[] } = await res.json();
20
21   // console.log("Data lapar yang diambil dari API:", response.data);
22   return {
23     props: {
24       lapars: response.data
25     }
26   }
27 }
28

```



2. Lakukan pengujian:

o Tambah data

Add a document

Parent path
/lapars

Document ID ⓘ
gJz7oEOyDCuEsoBjIUuh

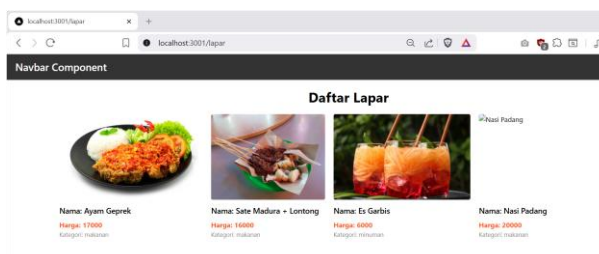
Field	Type	
name	string	⊖
String Nasi Padang		
image	string	⊖
String http		
price	number	20000 ⊖
category	string	⊖
String makanan		

⊕ Add field

Cancel Save

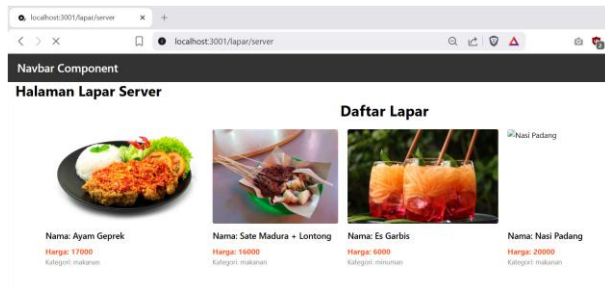
```
{
  "status": "true",
  "status_code": 200,
  "data": [
    {
      "id": "GduKwD9Gc7NcUr0u",
      "price": 17000,
      "image": "https://assets.unileversolutions.com/recipes-v3/297953.jpg",
      "category": "makanan",
      "name": "Ayam Geprek"
    },
    {
      "id": "09jK5TFFGjgqpuh91",
      "name": "Sate Madura + Lontong",
      "image": "https://thumbs.dreamstime.com/h/sate-madura-sate-madura-famous-throughout-indonesia-lontong-traditional-spices-28886529.jpg",
      "category": "makanan",
      "price": 16000
    },
    {
      "id": "7f6uEasDm57vdqg1",
      "price": 6000,
      "category": "minuman",
      "image": "https://idratech.co.id/wp-content/uploads/2022/04/Es-Garbis-1.jpg",
      "name": "Es Garbis"
    },
    {
      "id": "gJz7oEOyDCuEsoBjIUuh",
      "image": "http",
      "category": "makanan",
      "price": 20000,
      "name": "Nasi Padang"
    }
  ]
}
```

CSR /lapar



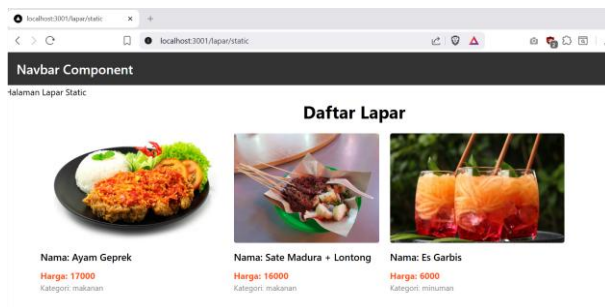
Makanan Nasi Padang langsung muncul dirender oleh Client

SSR /lapar/server



Makanan Nasi padang langsung muncul dirender oleh Server

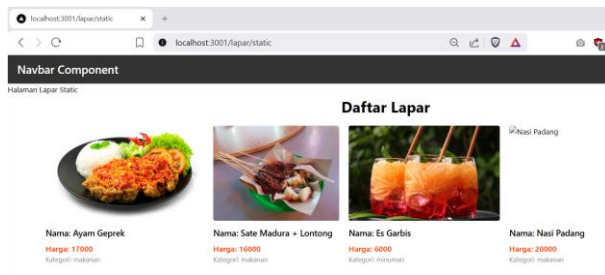
SSG /lapar/static



Makanan Nasi padang tidak langsung muncul pada static

Tapi setelah dilakukan restart pada npm run build lalu npm run start lagi

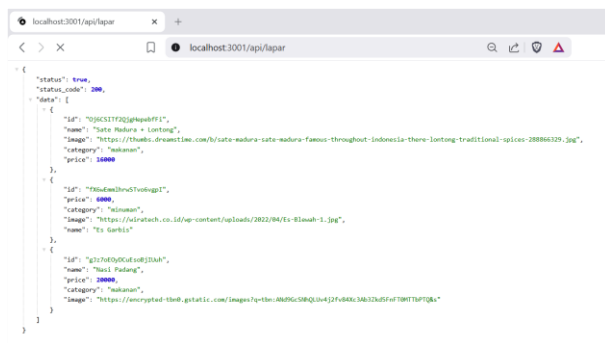
```
Route (pages)
├─ /
├─ /_app
├─ /404
├─ /about
├─ f /api/hello
├─ f /api/lapar
├─ f /api/produk
├─ o /auth/login
├─ o /auth/register
├─ o /blog
├─ o /blog/[slug]
├─ o /category/[[...slug]]
├─ o /lapar
├─ o /lapar/[id]
├─ f /lapar/server
├─ ● /lapar/static
├─ o /produk
├─ o /produk/[id]
├─ f /produk/server
├─ ● /produk/static
├─ o /profil
├─ o /profil/edit
├─ o /setting/app
├─ o /shop/[[...slug]]
├─ o /user
├─ o /user/password
```



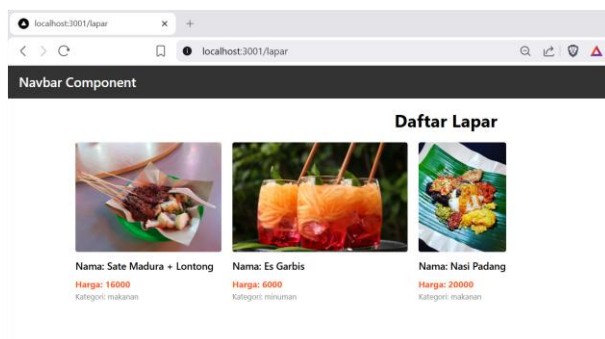
Makanan Nasi Padang muncul pada static.

o Hapus data

Hapus data Ayam Geprek

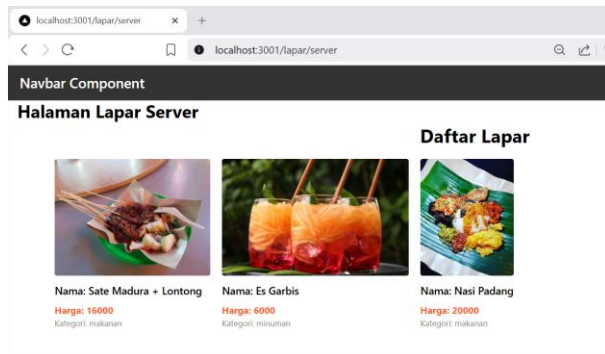


CSR



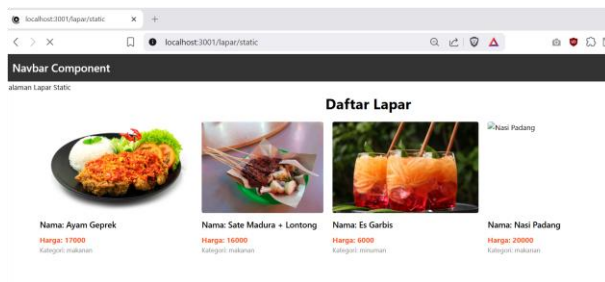
Makanan Ayam Geprek Langsung terhapus dari sisi Client.

SSR



Makanan Ayam Geprek langsung terhapus dari sisi Server.

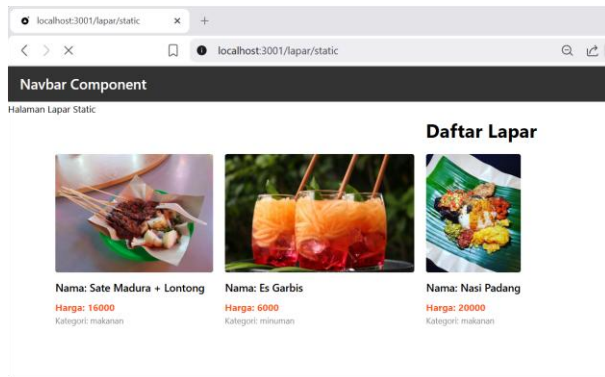
SSG



Makanan Ayam Geprek tidak langsung terhapus dari halaman Static.

Tapi setelah dilakukan restart pada npm run build lalu npm run start lagi

```
Route (pages)
├─ ○ /
├─ ○ /_app
├─ ○ /404
├─ ○ /about
├─ f /api/hello
├─ f /api/lapar
├─ f /api/produk
├─ ○ /auth/login
├─ ○ /auth/register
├─ ○ /blog
├─ ○ /blog/[slug]
├─ ○ /category/[[...slug]]
├─ ○ /lapar
├─ ○ /lapar/[id]
├─ f /lapar/server
├─ ● /lapar/static
├─ ○ /produk
├─ ○ /produk/[id]
├─ f /produk/server
├─ ● /produk/static
├─ ○ /profil
├─ ○ /profil/edit
├─ ○ /setting/app
├─ ○ /shop/[[...slug]]
├─ ○ /user
├─ ○ /user/password
```



Makanan Ayam Geprek Langsung terhapus dari halaman Static

o Bandingkan hasil

Fitur	CSR (/lapar)	SSR (/lapar/server)	SSG (/lapar/static)
Waktu Render	Sisi Client (Browser)	Sisi Server (Setiap Request)	Saat Build Time (Sekali saja)
Tambah Data	Langsung muncul. Data diambil saat halaman dimuat di browser.	Langsung muncul. Server mengambil data terbaru setiap kali halaman diakses.	Tidak langsung muncul. Data terkunci pada saat proses <i>build</i> terakhir.
Hapus Data	Langsung terhapus. UI melakukan sinkronisasi ulang dengan API.	Langsung terhapus. Server merender ulang halaman tanpa data tersebut.	Masih muncul. Halaman statis yang sudah jadi masih menyimpan data lama.
Update Data	Otomatis (Re-fetch di client)	Otomatis (Re-render di server)	Manual (Butuh npm run build)
Kecepatan	Cepat setelah <i>load</i> awal selesai.	Tergantung kecepatan server & API.	Paling Cepat (Melayani file HTML jadi).

Kesimpulan:

1. CSR (Client-Side Rendering): Perubahan data (tambah/hapus) langsung terlihat tanpa refresh karena data diambil langsung oleh browser.
2. SSR (Server-Side Rendering): Data selalu segar di setiap kunjungan karena server merender ulang halaman setiap kali ada permintaan.
3. SSG (Static Site Generation): Data terkunci saat proses build. Perubahan data tidak akan muncul sampai aplikasi dibangun ulang (rebuild).

3. Buat laporan analisis minimal 3 halaman.

E. STUDI ANALISIS

Jawab pertanyaan berikut:

1. Mengapa SSG tidak menampilkan data terbaru?
SSG mengambil data dan membuat file HTML statis hanya satu kali pada saat proses *build*, sehingga perubahan data setelahnya tidak akan terdeteksi hingga aplikasi di-*build* ulang.
2. Mengapa SSG lebih cepat?
SSG lebih cepat karena server hanya perlu mengirimkan file HTML yang sudah jadi (pra-render) langsung dari CDN tanpa harus memproses logika atau mengambil data dari database setiap kali ada permintaan.
3. Kapan SSG tidak cocok digunakan?
SSG tidak cocok digunakan pada halaman yang kontennya harus berubah secara *real-time* atau dipersonalisasi khusus untuk setiap pengguna (seperti keranjang belanja atau data stok yang sangat fluktuatif).
4. Mengapa e-commerce tidak cocok menggunakan SSG murni?
Karena informasi harga dan ketersediaan stok barang dapat berubah setiap saat, sementara SSG memerlukan *rebuild* manual untuk memperbarui informasi tersebut.
5. Apa perbedaan build mode dan development mode?
Development mode memprioritaskan fitur *hot-reloading* dan kemudahan *debugging* saat koding, sedangkan *build mode* melakukan optimasi penuh (minifikasi dan pra-render) untuk menghasilkan aplikasi yang siap dijalankan di lingkungan produksi dengan performa maksimal.

LAPORAN JOBSHEET 11: DYAMICROUTING MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

✂ C. LANGKAH PRAKTIKUM

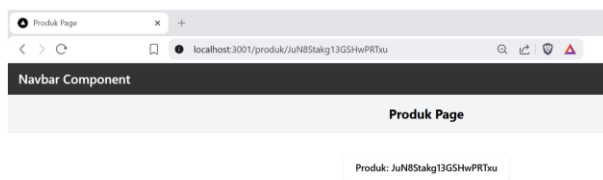
Bagian 1 – Membuat Dynamic Route

Buka file pages/products/[product].tsx dan modifikasi sbb (line 20)

```
perograman_hamawork - index.tsx
1 import styles from "../../pages/produk/produk.module.scss";
2 import Link from "next/link";
3 type ProductType = {
4   id: string;
5   name: string;
6   price: number;
7   image: string;
8   category: string;
9 };
10
11 const TampilanProduk = ({ products, isLoading }: { products: ProductType[], isLoading?: boolean }) => {
12   return (
13     <div className={styles.produk}>
14       <h3 className={styles.produk_title}>Daftar Produk</h3>
15       <div className={styles.produk_content}>
16         {isLoading ? (
17           <div>
18             {products.map((products: ProductType) => {
19               <Link href={`/produk/${products.id}`} key={products.id} className={styles.produk_content_item}>
20                 <div className={styles.produk_content_item} key={products.id}>
21                   <img src={products.image} alt={products.name} className={styles.produk_content_item_image} />
22                   <h3 className={styles.produk_content_item_name}>{products.name}</h3>
23                   <p className={styles.produk_content_item_price}>Harga Rp. {products.price.toLocaleString("id-ID")}</p>
24                   <p className={styles.produk_content_item_category}>Kategori: {products.category}</p>
25                 </div>
26               </Link>
27             )}
28           </div>
29         ) : (
30           <div>
31             {[1, 2, 3].map((item) => {
32               <div className={styles.produk_content_skeleton}>
33                 <div className={styles.produk_content_skeleton_image}></div>
34                 <div className={styles.produk_content_skeleton_name}></div>
35                 <div className={styles.produk_content_skeleton_category}></div>
36                 <div className={styles.produk_content_skeleton_price}></div>
37               </div>
38             )}
39           </div>
40         )}
41       </div>
42     </div>
43   );
44 }
45
46 export default TampilanProduk;
```

Jalankan browser <http://localhost:3000/produk>

o Jika kita klik salah satu gambar maka akan menuju halaman lain

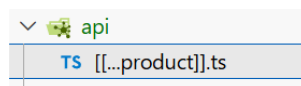


Bagian 2 – Implementasi CSR (Client Rendering)

o Modifikasi pada file [produk].tsx pada folder src/pages/produk/

```
pemograman_framework - [produk].tsx
1 import fetcher from "@utils/swr/fetcher";
2 import { useRouter } from "next/router";
3 import useSWR from "swr";
4
5 // Tambahan Langkah 4. Dynamic Routing gabung sama commit Langkah 3
6
7 const HalamanProduk = () => {
8   // const Router = useRouter();
9   // console.log(Router);
10  const { query } = useRouter();
11  const { data, error, isLoading } = useSWR(`/api/products/${query.produk}`, fetcher);
12  return (
13    <div>
14      <h1 className="text-2xl font-bold mb-4">Halaman Produk</h1>
15      <p className="text-lg">Produk: {query.produk}</p>
16    </div>
17  );
18 };
19
20 export default HalamanProduk;
21
22
```

Pada file produk.ts pada folder pages/api di rename menjadi [...product]].ts



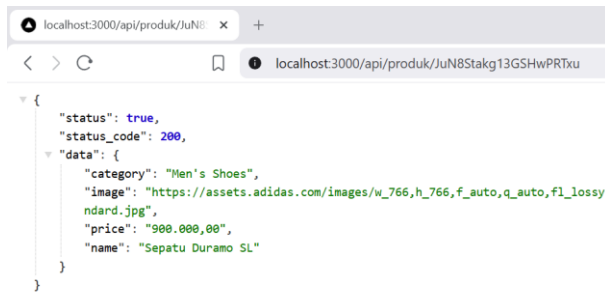
Modifikasi file servicefirebase.ts

```
pemograman_framework - servicefirebase.ts
1 import { getFirestore, collection, getDocs, Firestore, getDoc, doc } from 'firebase/firestore';
2 import app from './firebase';
3
4 const db = getFirestore(app);
5
6 export async function retrieveProducts(collectionName: string) {
7   const snapshot = await getDocs(collection(db, collectionName));
8   const data = snapshot.docs.map(doc => ({
9     id: doc.id,
10     ...doc.data()
11   }));
12   return data;
13 }
14
15 export async function retrieveDataById(collectionName: string, id: string) {
16   const snapshot = await getDoc(doc(db, collectionName, id));
17   const data = snapshot.data();
18   return data;
19 }
```

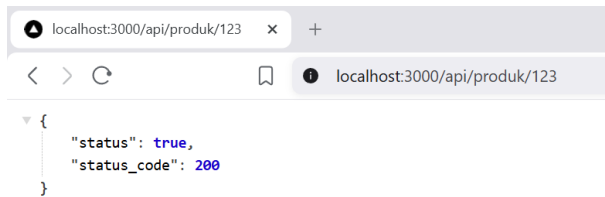
Modifikasi file [...produk]].ts

```
pemograman_framework - [...product]].ts
1 import { ProductType } from '../types/Product.type';
2 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
3 import type { NextApiRequest, NextApiResponse } from 'next';
4 import { retrieveDataById, retrieveProducts } from '../utils/servicefirebase';
5 type Data = {
6   status: boolean;
7   status_code: number;
8   data: any;
9 };
10 export default async function handler(
11   req: NextApiRequest,
12   res: NextApiResponse<Data>,
13 ) {
14   if (req.query.product![1]) {
15     const data = await retrieveDataById("products", req.query.ProductType![1]);
16     res.status(200).json({ status: true, status_code: 200, data: data });
17     return;
18   } else {
19     const data = await retrieveProducts("products");
20     res.status(200).json({ status: true, status_code: 200, data: data });
21   }
22 }
```

Jalankan browser <http://localhost:3000/api/produk/JuN8Stakg13GSHwPRTxu>



Jalankan alamat url <http://localhost:3000/api/produk/123>



Buat file dengan nama index.tsx pada folder views/DetailProduct selain itu buat juga

file dengan nama detailProduct.module.scss

Modifikasi detailProduct.module.scss

● ● ● pemograman_framework - detailProduct.module.scss

```
1  .produkdetail{
2    width: 80%;
3    margin: 40px auto;
4    padding: 24px;
5    display: flex;
6    gap: 40px;
7    background-color: #ffffff;
8    border-radius: 12px;
9    box-shadow: 0px 10px 25px rgba(0, 0, 0, 0.08);
10   align-items: flex-start;
11
12   &__image{
13     flex: 1;
14
15     img{
16       width: 100%;
17       border-radius: 12px;
18       object-fit: cover;
19     }
20   }
21
22   &__info{
23     flex: 1.2;
24     display: flex;
25     flex-direction: column;
26   }
27
28   &__name{
29     font-size: 28px;
30     font-weight: 700;
31     margin-bottom: 12px;
32     color: #222;
33   }
34
35   &__category{
36     font-size: 16px;
37     color: #666;
38     margin-bottom: 16px;
39   }
40
41   &__price{
42     font-size: 16px;
43     font-weight: bold;
44     color: #ff5722;
45     margin-bottom: 20px;
46   }
47
48   &__button{
49     width: fit-content;
50     padding: 12px 24px;
51     background-color: #111;
52     color: white;
53     border-radius: 8px;
54     cursor: pointer;
55     transition: 0.3s ease;
56
57     &:hover{
58       background-color: #ff5722;
59     }
60   }
61 }
62
63 /* Responsive */
64 @media (max-width: 768px) {
65   .produkDetail{
66     flex-direction: column;
67
68     &__image{
69       margin-bottom: 20px;
70     }
71   }
72 }
```

Modifikasi index.tsx pada folder DetailProduct

```
pemograman_framework - index.tsx
1 // import { ProductType } from "@types/Product.type";
2 import { ProductType } from "../../types/Product.type";
3 import styles from "../../DetailProduct/detailProduct.module.scss";
4
5 const DetailProduk = ({ products }: { products: ProductType }) => {
6   return (
7     <div className={styles.produkdetail}>
8       <div className={styles.produkdetail__image}>
9         <img src={products.image} alt={products.name} />
10      </div>
11
12      <div className={styles.produkdetail__info}>
13        <h1 className={styles.produkdetail__name}>{products.name}</h1>
14        <p className={styles.produkdetail__category}>{products.category}</p>
15        <p className={styles.produkdetail__price}>Rp{products.price.toLocaleString("id-ID")}</p>
16      </div>
17    </div>
18  );
19 }
20
21 export default DetailProduk;
```

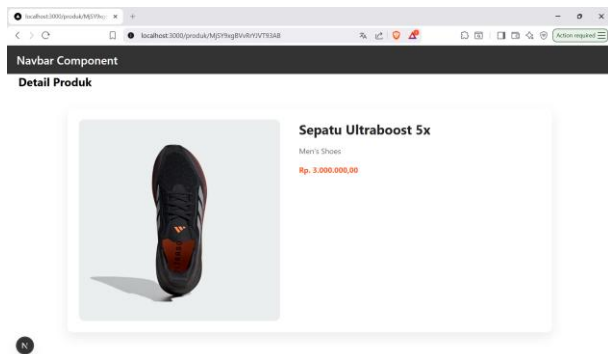
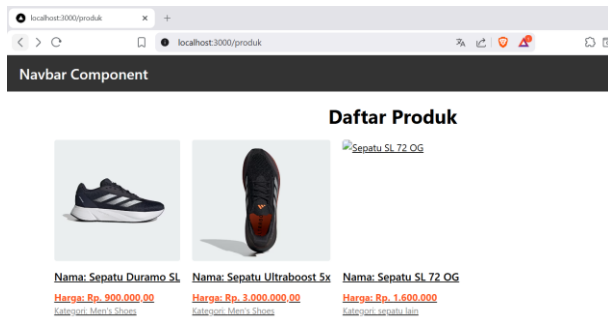
Modifikasi file [product].tsx

```
pemograman_framework - [produk].tsx
1 import fetcher from "@utils/swr/fetcher";
2 import { useRouter } from "next/router";
3 import useSWR from "swr";
4 import DetailProduk from "../../views/DetailProduct";
5
6 // Tambahan Langkah 4. Dynamic Routing gabung sama commit Langkah 3
7
8 const HalamanProduk = () => {
9   // const Router = useRouter();
10   // console.log(Router);
11   const { query } = useRouter();
12   const { data, error, isLoading } = useSWR(`/api/products/${query.produk}`, fetcher);
13   return (
14     <div>
15       <DetailProduk products={isLoading ? [] : data.data} />
16     </div>
17   );
18 };
19
20 export default HalamanProduk;
```

Modifikasi index.tsx pada folder views/detailProduct line 16

```
pemograman_framework - index.tsx
1 // import { ProductType } from "@types/Product.type";
2 import { ProductType } from "../../types/Product.type";
3 import styles from "../../DetailProduct/detailProduct.module.scss";
4
5 const DetailProduk = ({ products }: { products: ProductType }) => {
6   return (
7     <div>
8       <h1 className="h1-6 text-2xl font-bold">Detail Produk</h1>
9       <div className={styles.produkdetail}>
10         <div className={styles.produkdetail__image}>
11           <img src={products.image} alt={products.name} />
12         </div>
13
14         <div className={styles.produkdetail__info}>
15           <h1 className={styles.produkdetail__name}>{products.name}</h1>
16           <p className={styles.produkdetail__category}>{products.category}</p>
17           <p className={styles.produkdetail__price}>Rp. {products.price && products.price.toLocaleString("id-ID")}</p>
18         </div>
19       </div>
20     </div>
21   );
22 }
23
24 export default DetailProduk;
```

Jalankan browser <http://localhost:3000/produk/> saat produk diklik maka akan muncul detailProduk <http://localhost:3000/produk/JuN8Stakg13GSHwPRTxu>



Agar tulisan detail produk ditengah maka modifikasi file detailProduct.module.scss line 103-108 dan file index.tsx tambahkan code pada line 7,8 dan 22 menjadi

```

1 // import { ProductType } from "B/types/Product.type";
2 import { ProductType } from "B/types/Product.type";
3 import styles from "B/detailProduct/detailProduct.module.scss";
4
5 const DetailProduk = ({ products }: { products: ProductType }) => {
6   return (
7     <div>
8       <h1 className={styles.title}>Detail Produk</h1>
9       <div className={styles.produkdetail}>
10         <div className={styles.produkdetail_image}>
11           <img src={products.image} alt={products.name} />
12         </div>
13         <div className={styles.produkdetail_info}>
14           <h1 className={styles.produkdetail_name}>{products.name}</h1>
15           <p className={styles.produkdetail_category}>{products.category}</p>
16           <p className={styles.produkdetail_price}>Rp. {products.price && products.price.toLocaleString("id-ID")}</p>
17         </div>
18       </div>
19     </div>
20   )
21 }
22
23
24 export default DetailProduk;

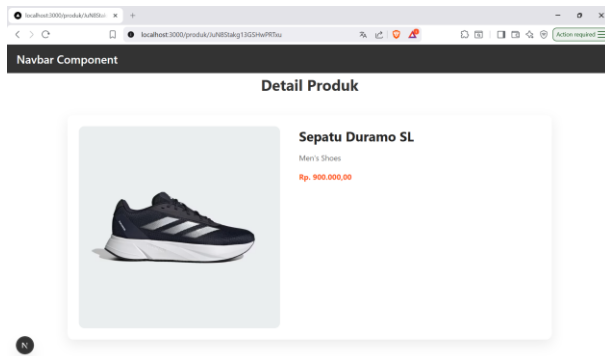
```

```

1 pemograman_framework - detailProduct.module.scss
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74 .title{
75   text-align: center;
76   font-size: 32px;
77   font-weight: bold;
78   margin-bottom: 40px;
79   color: #333;
80 }

```

Sehingga hasilnya seperti berikut



Bagian 3 – Implementasi SSR

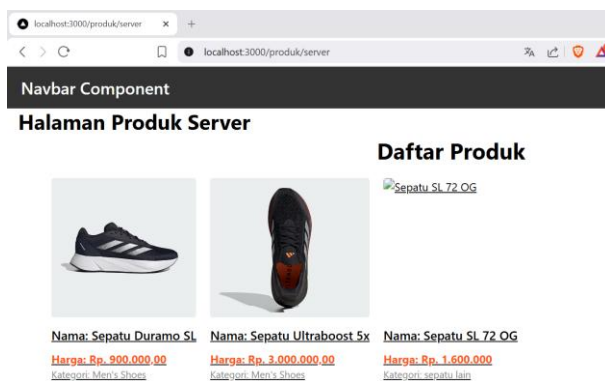
Modifikasi [produk].tsx pada folder src/pages/produk dan comment line 9 sampai 20
dikarena kita akan menggunakan metode SSR. Tambahkan beberapa kode untuk SSR

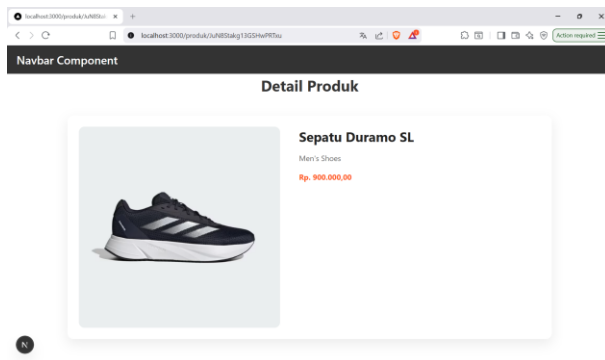
```

1 import { fetcher } from '@utils/swr/fetcher';
2 import { useRouter } from 'next/router';
3 import useSWR from 'swr';
4 import DetailProduk from '../views/DetailProduk';
5 import { ProductType } from '@types/Product.type';
6
7 const HalamanProduk = ({product}: {product: ProductType}) => {
8   // (digunakan client-side rendering)
9   // // const Router = useRouter();
10  // // console.log(Router);
11  // const { query } = useRouter();
12  // const {data, error, isLoading} = useSWR('/api/products/${query.produk}', fetcher);
13  // return (
14  //   <div>
15  //     <DetailProduk products={isLoading ? [] : data.data} />
16  //   </div>
17  // );
18
19  return (
20    <div>
21      <DetailProduk products={product} />
22    </div>
23  );
24 };
25
26 export default HalamanProduk;
27
28 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
29 // (digunakan server-side rendering)
30 export async function getServerSideProps({params}: {params: {produk: string}}) {
31   const res = await fetch('http://localhost:3000/api/products/${params.produk}');
32   const response = await res.json();
33   console.log("Data produk yang diambil dari API:", response);
34   return {
35     props: {
36       product: response.data, //Pastikan untuk memberikan nilai default jika data tidak tersedia
37     }
38   }
39 }
40
41
42

```

Jalankan browser <http://localhost:3000/produk/server>





Note: Tidak perlu loading state karena data sudah tersedia sebelum render.

Bagian 4 – Implementasi Static Site Generation (Dynamic SSG)

Buka file [produk].tsx dan modifikasi seperti berikut

```

1 import { useRouter } from 'next/router';
2 import { useRouter } from 'next/router';
3 import useSWR from 'swr';
4 import DetailProduk from '../views/DetailProduct';
5 import { ProductType } from '@types/Product.type';
6
7 const halamanProduk = ({product}: {product: ProductType}) => {
8   // (Digunakan client-side rendering)
9   // // const Router = useRouter();
10  // // console.log(Router);
11  // const { query } = useRouter();
12  // const { data, error, isLoading } = useSWR('/api/products/${query.produk}', fetcher);
13  // return (
14  //   <div>
15  //     <DetailProduk products={isLoading ? [] : data.data} />
16  //   </div>
17  // );
18
19  return (
20    <div>
21      <DetailProduk products={product} />
22    </div>
23  );
24 };
25
26 export default halamanProduk;
27
28 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
29 // (Digunakan server-side rendering)
30 // export async function getServerSideProps({params}: {params: {produk: string}}) {
31 //   const res = await fetch('http://localhost:3000/api/products/${params.produk}');
32 //   const response = await res.json();
33 //   // console.log('Data produk yang diambil dari API:', response);
34 //   return {
35 //     props: {
36 //       product: response.data, //Pastikan untuk memberikan nilai default jika data tidak tersedia
37 //     }
38 //   };
39 // }
40
41 // (Digunakan static-site generation)
42 export async function getStaticPaths() {
43   const res = await fetch('http://localhost:3000/api/products');
44   const response = await res.json();
45
46   const paths = response.data.map((product: ProductType) => ({
47     params: { produk: product.id },
48   }));
49
50   // console.log('Paths yang dihasilkan untuk produk:', paths); // Debugging: Tampilkan paths yang dihasilkan
51   return {
52     paths,
53     fallback: false,
54   };
55 }
56
57 export async function getStaticProps({params}: {params: {produk: string}}) {
58   const res = await fetch('http://localhost:3000/api/products/${params.produk}');
59   // const response: ProductType[] = await res.json();
60   const response = { data: ProductType[] } = await res.json();
61
62   console.log('Data produk yang diambil dari API:', response);
63   return {
64     props: {
65       product: response.data,
66     }
67   };
68 }
69
70
71

```

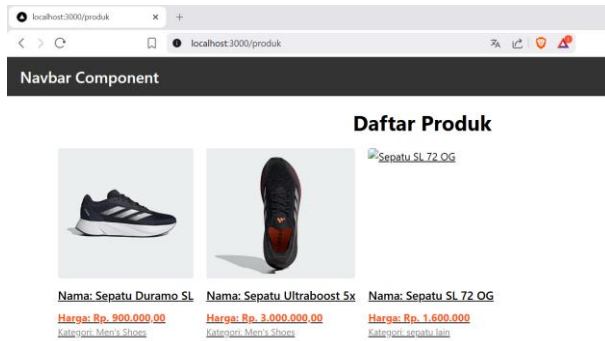
Buka file index.tsx pada folder src/views/DetailProduct dan modifikasi pada line 11

```

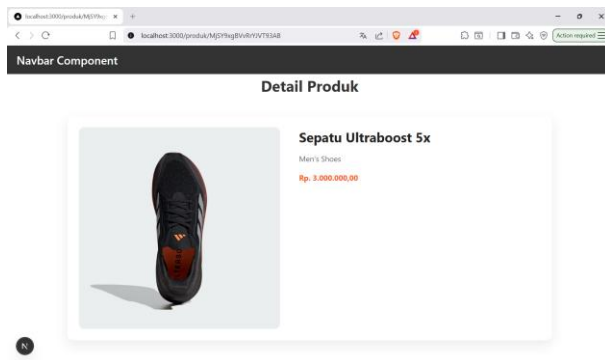
11 <img src={products.image} alt={products.name} />

```

Jalankan browser <http://localhost:3000/produk>

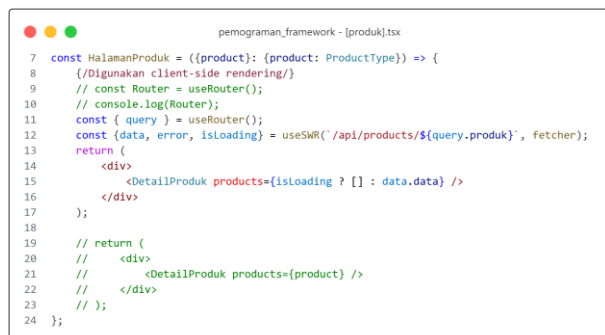


Saat diklik salah satu produk

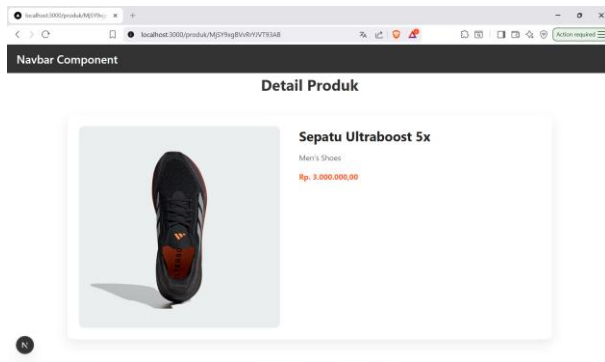


D. PENGUJIAN

Uji 1 – CSR



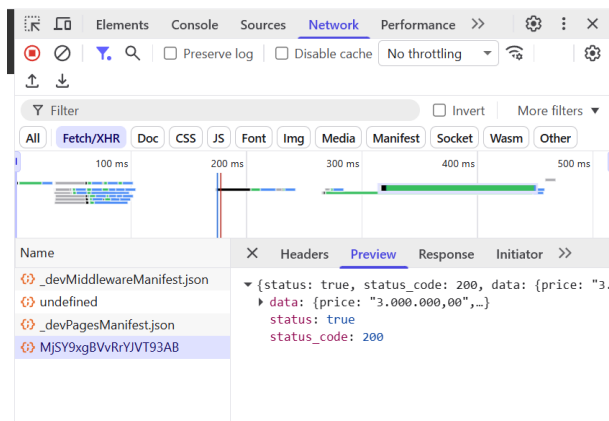
- Refresh halaman detail



- Perhatikan loading

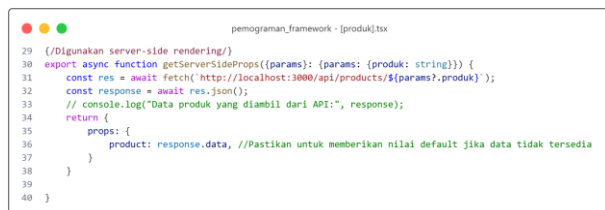
Ada loadingnya

- Periksa Network → XHR → API request terlihat

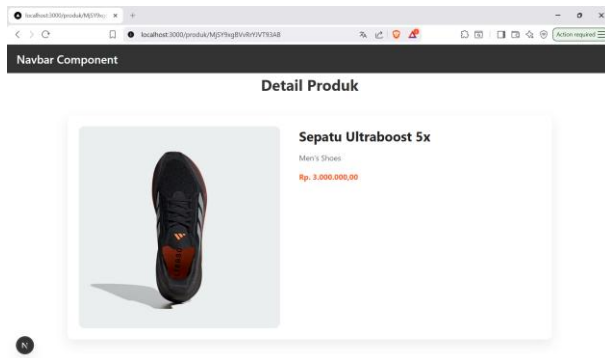


Data dirender pada client side/browser, bisa dilihat API ada pada gambar Fetch/XHR diatas.

Uji 2 – SSR



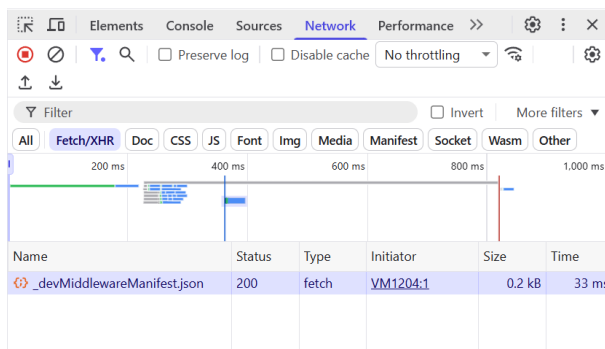
- Refresh halaman detail



- Tidak ada loading

Benar

- Periksa Network → tidak terlihat fetch detail



Benar, karena halaman dirender pada server bukan di browser.

Uji 3 – SSG (Lakukan seperti langkah sebelumnya pada Jobsheet 10)

1. Jalankan:

o npm run build

Route (pages)

- o /
- o /_app
- o /404
- o /about
- f /api/[[...produk]]
- f /api/hello
- f /api/lapar
- o /auth/login
- o /auth/register
- o /blog
- o /blog/[slug]
- o /category/[[...slug]]
- o /lapar
- o /lapar/[id]
- f /lapar/server
- /lapar/static
- o /produk
- f /produk/[produk]
- f /produk/server
- /produk/static
- o /profil
- o /profil/edit
- o /setting/app
- o /shop/[[...slug]]
- o /user
- o /user/password

o npm run start

○ PS D:\pemograman_framework\Code\praktikum\my-app> npm run start

```
> my-app@0.1.0 start
> next start
```

▲ Next.js 16.1.6

```
- Local:      http://localhost:3000
- Network:    http://172.16.0.2:3000
```

✓ Starting...

✓ Ready in 396ms

2. Tambahkan produk baru di database.

Add a document

Parent path
/products

Document ID ⓘ
9NX0vNtnhSsMin5w7ZtG

FieldType

name=string

String

Sepatu Running Runfalcon 5

FieldTypeValue

price=number850000

FieldType

image=string

String

https://assets.adidas.com/images/h_2000,f_auto,q_auto,fl_lossy,c_fill,g_auto/6e51a95a07f9474cbb7361ba75eecc41_9366/Sepatu_Running_Runfalcon_5_Putih_JQ9402_HM4.jpg

FieldType

category=string

String

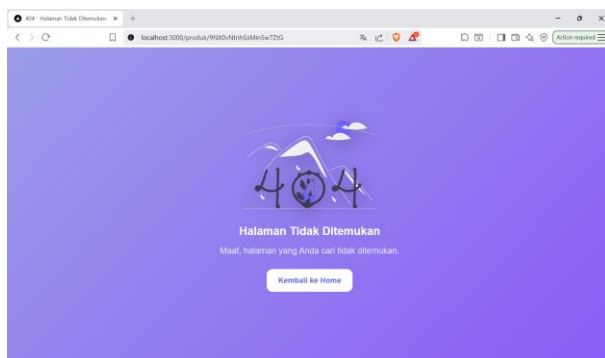
Sepatu Lari

Add field

CancelSave

3. Buka halaman detail produk baru:

<http://localhost:3000/produk/9NX0vNtnhSsMin5w7ZtG>



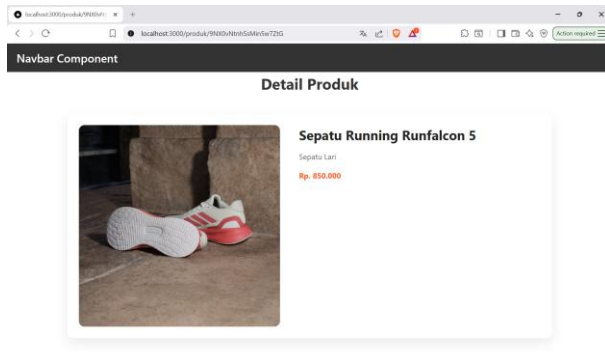
Tidak muncul.

Benar

4. Build ulang:

o npm run build

o npm run start



Baru muncul.

Benar.

E. TUGAS PRAKTIKUM

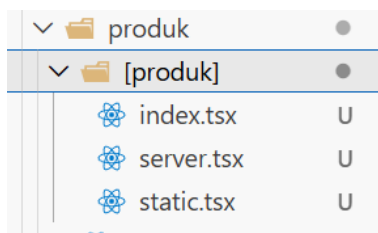
Tugas Individu

1. Implementasikan halaman detail dengan:

o CSR

o SSR

o SSG



Tambah data baru

Add a document

Parent path
/products

Document ID ⓘ
JWICADTeqfM4i6lbYftY

Field

name

=

Type

string

⊖

String
SUPERSTAR II PAPUA

Field

price

=

Type

number

Value

2200000

⊖

Field

image

=

Type

string

⊖

String
https://assets.adidas.com/images/h_2000,f_auto,q_auto,fl_lossy,c_fill,g_auto/7081eeb6184b4bcea7c4471a3bb3c564_9366/SUPERSTAR_II_PAPUA_Krem_KJ8082_04_standard.jpg

Field

category

=

Type

string

⊖

String
Eksklusif di Indonesia | Stok Terbatas

⊕ Add field

Cancel

Save

Lalu build dan start

build

Route (pages)

```
├─ ○ /
├─   /_app
├─ ○ /404
├─ ○ /about
├─ f /api/[[...produk]]
├─ f /api/hello
├─ f /api/lapar
├─ ○ /auth/login
├─ ○ /auth/register
├─ ○ /blog
├─ ○ /blog/[slug]
├─ ○ /category/[[...slug]]
├─ ○ /lapar
├─ ○ /lapar/[id]
├─ f /lapar/server
├─ ● /lapar/static
├─ ○ /produk
├─ ○ /produk/[produk]
├─ f /produk/[produk]/server
├─ ● /produk/[produk]/static (1036 ms)
├─   /produk/9NX0vNtnhSsMin5w7ZtG/static
├─   /produk/JuN8Stakg13GSHwPRTxu/static
├─   /produk/MjSY9xgBVvRrYJVT93AB/static
├─   /produk/nuggh8wLOzGCjAmcRvqP/static
├─ f /produk/server
├─ ● /produk/static
├─ ○ /profil
├─ ○ /profil/edit
├─ ○ /setting/app
├─ ○ /shop/[[...slug]]
├─ ○ /user
├─ ○ /user/password
```

Start

```
○ PS D:\pemograman_framework\Code\praktikum\my-app> npm run start

> my-app@0.1.0 start
> next start

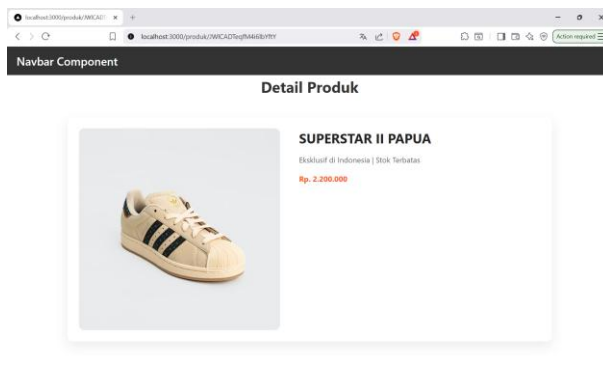
▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Network:    http://172.16.0.2:3000

✓ Starting...
✓ Ready in 405ms
```

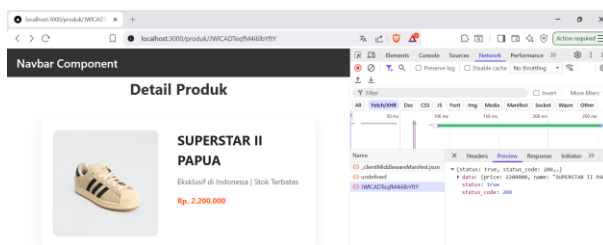
CSR

```
pemograman_framework - index.tsx

1 import fetcher from "@utils/swr/fetcher";
2 import { useRouter } from "next/router";
3 import useSWR from "swr";
4 import DetailProduk from "../../views/DetailProduct";
5 import { ProductType } from "@types/Product.type";
6
7 const HalamanProduk = ({product}: {product: ProductType}) => {
8   // (Digunakan client-side rendering/)
9   // const Router = useRouter();
10  // console.log(Router);
11  const { query } = useRouter();
12  const {data, error, isLoading} = useSWR(`/api/products/${query.product}`, fetcher);
13  return (
14    <div>
15      <DetailProduk products={isLoading ? [] : data.data} />
16    </div>
17  );
18 };
19
20 export default HalamanProduk;
```



Bisa langsung tampil produknya dengan loading

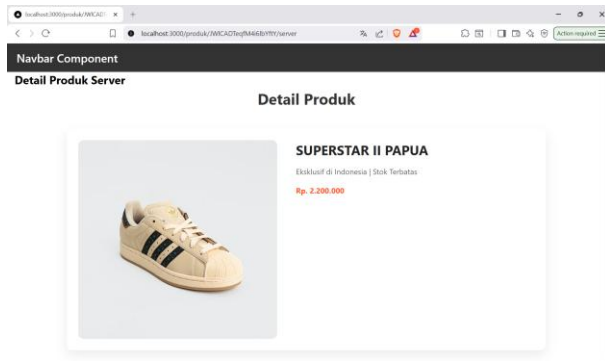


Request API terlihat pada network.

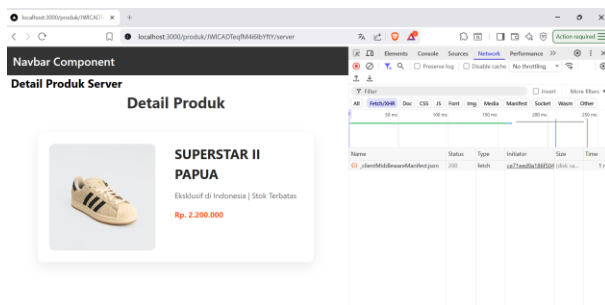
SSR

```
pemograman_framework - server.tsx

1 import DetailProduk from "../../views/DetailProduct";
2 import { ProductType } from "@types/Product.type";
3
4 const HalamanProdukServer = ({product}: {product: ProductType}) => {
5   return (
6     <div className="text-2xl font-bold ml-4">Detail Produk Server</div>
7     <div>
8       <DetailProduk products={product} />
9     </div>
10  );
11 };
12
13 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
14 // (Digunakan server-side rendering)
15 export async function getServerSideProps({params}: {params: {product: string}}) {
16   const res = await fetch('http://localhost:3000/api/products/${params.product}');
17   const response = await res.json();
18   // console.log("Data produk yang diambil dari API:", response);
19   return {
20     props: {
21       product: response.data, //Pastikan untuk memberikan nilai default jika data tidak tersedia
22     }
23   };
24 }
25
26 export default HalamanProdukServer;
```

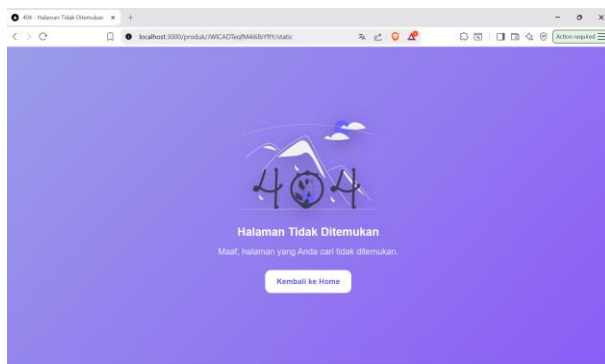


Bisa langsung tampil produknya tanpa ada loading



Request API tidak terlihat pada network.

SSG



Produk yang baru ditambahkan di database tidak bisa langsung terlihat, sehingga perlu build dan start ulang.

```

Route (pages)
├─ /
├─ /_app
├─ /404
├─ /about
├─ f /api/[[...produk]]
├─ f /api/hello
├─ f /api/lapar
├─ o /auth/login
├─ o /auth/register
├─ o /blog
├─ o /blog/[slug]
├─ o /category/[[...slug]]
├─ o /lapar
├─ o /lapar/[id]
├─ f /lapar/server
├─ ● /lapar/static (306 ms)
├─ o /produk
├─ o /produk/[produk]
├─ f /produk/[produk]/server
├─ ● /produk/[produk]/static (858 ms)
├─ | /produk/9NX0vNtnhSsMin5w7ZtG/static (306 ms)
├─ | /produk/Jw1CADTeqfM4i6lbYftY/static
├─ | /produk/JuN8Stakg13GSHwPRTxu/static
├─ | /produk/MjSY9xgBVvRrYJVT93AB/static
├─ | /produk/nuggh8wLOzGCjAmcRvqP/static
├─ f /produk/server
├─ ● /produk/static (306 ms)
├─ o /profil
├─ o /profil/edit
├─ o /setting/app
├─ o /shop/[[...slug]]
├─ o /user
├─ o /user/password

```

PS D:\pemograman_framework\Code\praktikum\my-app> npm run start

```

> my-app@0.1.0 start
> next start

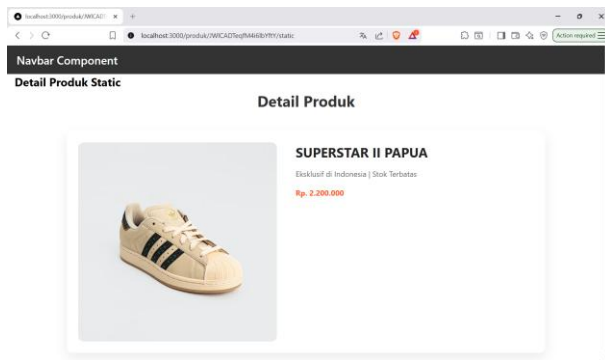
▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Network:    http://172.16.0.2:3000

```

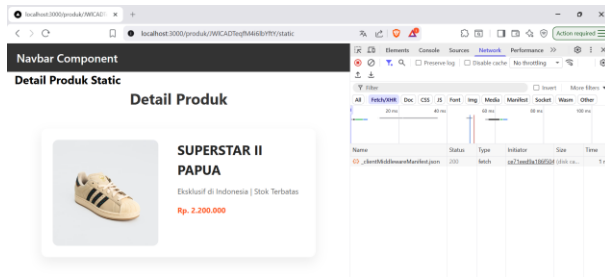
```

✓ Starting...
✓ Ready in 398ms

```



Setelah itu produk baru bisa tampil



Dan pada network tidak terlihat API dari data produk yang tertampil, karena data sudah diburn ke html nya langsung.

2. Buat tabel perbandingan:

Aspek	CSR	SSR	SSG
Loading	Ada loading , data diambil setelah halaman muncul di browser.	Tidak ada loading , halaman datang sudah berisi data	Tidak ada loading , halaman sudah jadi file HTML statis.
Build Required	Tidak perlu , cukup deploy, perubahan kode tetap butuh build.	Tidak perlu , data diambil setiap ada request baru.	Perlu , harus build ulang jika ada data baru di database.
SEO	Kurang , karena konten kosong saat pertama dimuat.	Bagus , karena konten lengkap dibaca crawler.	Bagus , konten sudah diburn di HTML.
Perubahan Data	Langsung terlihat , real-time via fetch/SWR.	Langsung terlihat , fetch ulang setiap request.	Tidak langsung terlihat , butuh build & start ulang untuk data baru.

3. Dokumentasikan:

- o Screenshot
- o Network tab
- o Build result

Jawaban pada nomor 1 diatas.

F. PERTANYAAN ANALISIS

1. Mengapa `getStaticPaths` wajib pada dynamic SSG?

agar Next.js dapat mengetahui daftar semua ID yang ada untuk di-generate menjadi file HTML statis pada saat proses *build*.

2. Mengapa CSR membutuhkan loading state?

Karena browser baru akan mengambil data dari API setelah halaman HTML kosong selesai dimuat, sehingga ada jeda waktu tunggu bagi pengguna.

3. Mengapa SSG tidak menampilkan produk baru tanpa build ulang?

karena datanya sudah diburn menjadi file HTML statis saat proses *build*, sehingga perubahan di database tidak akan merubah file fisik yang sudah jadi tersebut.

4. Mana metode terbaik untuk halaman detail e-commerce?

Metode terbaik adalah SSR (atau ISR) karena memungkinkan konten produk terindeks SEO dengan sempurna dan data harga atau stok selalu diperbarui setiap kali halaman diakses.

5. Apa risiko menggunakan SSG untuk produk yang sering berubah?

Ketidaksinkronan data (informasi lama), di mana pelanggan melihat harga atau stok lama yang sudah tidak sesuai dengan kondisi asli di database.